

MODULE 6

Rapid Prototyping & Design Sprints

Renaissance Developer Academy — Detailed Lesson Plan

Week 6 of 10 | 5 Days | Full-Time Intensive
Badge Earned: Rapid Builder

Prerequisite: Module 5 — Reliable Systems Engineer
DRAFT — March 2026

Module Overview

The first five modules built individual capability: mindset (M1), verification (M2), building (M3), design (M4), and reliability (M5). Module 6 is where the program **shifts to team impact**. This week simulates the forward-deployed engineering reality: ambiguous problems, time pressure, cross-functional teams, and the demand to deliver working solutions in hours, not weeks.

The week centers on a **48-hour hackathon** (Tuesday–Wednesday) using design sprint methodology. Students form teams, select problems from a curated bank of real-world business challenges, and prototype solutions using the full AI toolset. In parallel, students build and orchestrate **ClawSwarm multi-agent systems** — the third step in the OpenClaw ecosystem progression (NanoBot → NanoClaw → ClawSwarm). The week ends with user validation sessions against real stakeholders and a Demo Day where prototypes are judged on business impact, not just technical elegance.

Connection to the JD: This module maps directly to Rapid Prototyping & Validation (“shape culture around rapid validation and iterative delivery, define standards for prototype-to-production, exemplify the deliver-in-days mindset”), Problem Discovery (“transform ambiguous situations into clear opportunities”), and Team Collaboration (“transform dysfunctional team dynamics, build exceptional teams”).

Learning Objectives

1. Apply design sprint methodology (define → ideate → prototype → test) under real time pressure to build working solutions in 48 hours
2. Orchestrate multi-agent systems using ClawSwarm: configure specialized agents, unified messaging, and role-based coordination
3. Lead cross-functional rapid delivery: scope management, task decomposition, iterative delivery, and team coordination under pressure
4. Conduct structured user validation sessions: prepare interview guides, test with real stakeholders, and synthesize findings
5. Practice the Problem Discovery skill: transform ambiguous business challenges into concrete, buildable problem statements
6. Navigate the tension between speed and quality: when to take technical debt intentionally and when to invest in doing it right

Pre-Work (Assigned End of Module 5)

Due before Day 1. Estimated time: 2–3 hours.

1. **Reading:** Read “The Lean Startup” Chapters 1–4 (or summary) focusing on Build-Measure-Learn, the MVP concept, and validated learning. Upload to NotebookLM and generate an Audio Overview.

2. **ClawSwarm Reconnaissance:** Clone the ClawSwarm repository from GitHub. Read the README, architecture documentation, and the gRPC gateway section. Understand: how does ClawSwarm differ from Nanobot and NanoClaw? What is a Hierarchical Swarm?
3. **Hackathon Prep:** Review the problem bank (distributed before Module 6). Rank your top 3 problem choices. Think about what kind of team you'd want to work with — complementary skills, not identical ones.
4. **User Testing Basics:** Read Steve Krug's "Rocket Surgery Made Easy" Chapter 1 (10-min summary of usability testing). Key concept: watch users use your product, don't ask them what they want.

Week-at-a-Glance

Day	Theme	Primary Tools	Key Output
Day 1	Design Sprint: Problem Discovery & ClawSwarm Setup	ClawSwarm, Kiro, whiteboard	Problem statement, team formed, ClawSwarm running
Day 2	48-Hour Hackathon: Build Sprint Day 1	All AI tools, GitHub Actions	Working prototype core, multi-agent config
Day 3	48-Hour Hackathon: Build Sprint Day 2 + User Testing	All tools, user test participants	User-tested prototype, validation findings
Day 4	Iteration, Polish & Multi-Agent Deep Dive	ClawSwarm, Claude Code, all tools	Iterated prototype, ClawSwarm multi-agent system
Day 5	Demo Day, Retrospective & Calibration	All tools, NotebookLM	Final demo, retro report, development map v6

Day 1: Design Sprint — Problem Discovery & ClawSwarm Setup

Theme: Transform ambiguous business challenges into buildable problem statements, form high-performing teams, and set up multi-agent infrastructure

Goal: Every team has a sharp problem statement, a scoped solution concept, a task breakdown, and a working ClawSwarm multi-agent system ready to support tomorrow's build sprint.

Schedule

09:00	Week 6 Opening: The Shift to Team Delivery	30 min
	- Module 6 marks the program's pivot from individual capability to team impact - The Renaissance Developer doesn't just build alone — they lead rapid delivery for teams - This week's arc: discover a problem (today), prototype a solution (Tue–Wed), validate with users (Wed), iterate (Thu), present (Fri) - Hackathon rules: teams of 3–4, all AI tools available, must deploy to production, must test with real users - Connecting to the JD: “shape culture around rapid validation, deliver in days, transform ambiguous situations into clear opportunities”	
09:30	Problem Discovery Workshop	90 min
	- EXERCISE 1: From Ambiguity to Clarity - The instructor presents 6–8 problem statements from a curated problem bank (see Appendix C): — Each problem is deliberately ambiguous — a real business pain, not a technical spec — Examples: “Our sales team wastes 3 hours/day updating CRM after customer calls” — “New hires take 6 weeks to become productive because institutional knowledge is scattered” — “Manufacturing quality issues are caught too late because inspection data lives in spreadsheets” - Problem Discovery process (practiced individually, then in teams): — Step 1: Restate the problem from the USER's perspective (not the technologist's) — Step 2: Identify the core pain — what is the actual cost (time, money, frustration) of this problem? — Step 3: Define the boundary — what is IN scope for a 48-hour prototype? What is OUT? — Step 4: Write a problem statement: “[User] needs [capability] because [pain point], and today they [current workaround]”	

	— Step 5: Define the MVP — what is the smallest thing you could build that would prove the solution works? - Students vote on top 3–4 problems. Teams form around problems (not the other way around).	
11:00	Break	15 min
	Teams finalize membership	
11:15	Team Formation & Solution Design Sprint	75 min
	- EXERCISE 2: Team Solution Design (in newly formed teams of 3–4) - Each team has the same structured process: — Round 1 — Silent Sketching (15 min): each team member independently sketches their solution concept on paper (not code, not AI — pencil and paper) — Round 2 — Present & Vote (15 min): each person presents their sketch (2 min). Team votes on strongest elements from each concept. — Round 3 — Merge & Scope (20 min): combine the best ideas into one solution. Write the team’s problem statement, solution description, and 5–8 user stories. — Round 4 — Task Breakdown (15 min): decompose into tasks for tomorrow’s build sprint. Assign ownership. Define what “done” looks like for each task. — Round 5 — Tech Stack Decision (10 min): what technologies, which AI tools for which parts, who builds what? - Commit the team’s problem statement, solution design, and task breakdown to a new team GitHub repository - Set up GitHub Projects board: Backlog → Today → In Progress → Review → Done	
12:30	Lunch	60 min
	Team lunch — start bonding	
13:30	ClawSwarm: Multi-Agent Architecture & Setup	90 min
	- EXERCISE 3: Set up ClawSwarm for team-based multi-agent development - ClawSwarm architecture overview (from pre-work reading, now hands-on): — The gRPC Gateway: unified message ingestion from Telegram, Discord, WhatsApp — Hierarchical Swarm: an orchestrator agent that routes tasks to specialized worker agents — Worker models: each worker can use a different LLM optimized for its role — Unified messaging: all agents communicate through a single normalized schema - Team setup exercise: — Clone ClawSwarm and follow the setup wizard (claw_config.yaml) — Configure at least 2 specialized agents for your team’s project:	

	• Agent A — Research/Planning: summarize requirements, suggest approaches, draft specs • Agent B — Developer: write code, run tests, create PRs (connected to your team's repo) • (Optional) Agent C — Reviewer: analyze code quality, suggest improvements, check for bugs • — Connect to at least 1 messaging platform (team Discord/Slack/Telegram channel) • — Test: send a task to the team channel, verify the orchestrator routes it to the correct agent • — Deploy ClawSwarm using Docker (apply Module 3–4 container skills) • Key architectural insight: ClawSwarm's Hierarchical Swarm pattern mirrors real team management — an orchestrator (tech lead) delegates to specialists • Compare with Module 3's Antigravity Manager View: ClawSwarm is messaging-first, Antigravity is IDE-first. When would you choose each?
--	---

15:00	Break	15 min
	• Team check-in	

15:15	Hackathon Preparation & Architecture Quick-Design	60 min
	• EXERCISE 4: Team Architecture Sprint • — Quick architecture sketch for the prototype (whiteboard or Kiro spec — keep it lean, not Module 4-level) • — Key decision: what corners can you cut for a 48-hour prototype? What MUST be solid? • — Write 1 ADR: your most important trade-off decision (“We chose speed over X because Y”) • — Set up CI/CD pipeline for the team repo (reuse Module 3 patterns — lint, test, build, deploy) • — Each team member should have at least 1 commit by end of day • Instructor circuit: brief check-in with each team on scope, architecture, and team dynamics • — Key question: “Is this achievable in 48 hours? What would you cut first if you're behind schedule?”	

16:15	Day 1 Wrap-Up	30 min
	• Each team: 2-minute pitch of their problem and solution concept to the cohort • Peer reactions: “What excites you? What concerns you?” (quick, unstructured feedback) • HOMEWORK: Finalize your task assignments. Set up your personal development branch. Get a good night's sleep — tomorrow is intense. • Preview: 48-hour hackathon starts at 09:00 sharp tomorrow	

Instructor Notes — Day 1: Day 1 requires careful facilitation of team formation. The ideal team has 3–4 members with complementary skills (one strong on frontend, one on backend, one on

design/UX, one who can flex). If students self-select into homogeneous teams (all backend engineers), intervene gently. The Problem Discovery Workshop is the most important exercise — teams that define sharp problem statements on Day 1 build better prototypes on Days 2–3. Watch for scope creep during the Solution Design Sprint: teams consistently under-estimate what 48 hours means and over-scope. Push them to cut: “What is the ONE thing your prototype must do to prove the solution works?” The ClawSwarm setup may hit snags with gRPC dependencies — have a troubleshooting guide ready. Teams that get ClawSwarm working will have a powerful tool for the hackathon; teams that struggle should fall back to Antigravity Manager View.

Day 2: 48-Hour Hackathon — Build Sprint Day 1

Theme: Heads-down building with AI tools at maximum speed, applying everything learned in Modules 1–5

Goal: Every team has the core of their prototype working by end of day: backend functional, frontend connected, at least 1 key user flow complete end-to-end.

Schedule

09:00	Hackathon Kickoff	15 min
	- The clock starts now. 48 hours until Demo Day judging on Thursday morning. - Ground rules: — Use ALL available AI tools — this is not about writing every line by hand — But VERIFY everything — your Module 2 AI Code Review Checklist applies even at hackathon speed — Commit frequently with descriptive messages (at least every 30 minutes per team member) — CI must be green at all times — never break main — Deploy to staging at least 3 times today — early and often — Keep your GitHub Projects board updated — tasks move across columns in real-time - Milestones: 11:00 (backend working), 14:00 (frontend connected), 16:00 (1 complete user flow)	
09:15	Build Sprint Block 1: Foundation	165 min
	- EXERCISE 5: Build the prototype core - Recommended parallel workflow for a team of 3–4: — Person A: Backend API + database (Claude Code CLI for scaffolding, Kiro for spec alignment) — Person B: Frontend shell + navigation (Antigravity for rapid UI, or Claude Code + framework) — Person C: Key feature logic + integration (whatever tool fits the problem best) — Person D (if 4-person team): ClawSwarm coordination, testing, CI/CD, deployment - Use ClawSwarm to support the build: — Send requests to specialized agents via team messaging channel — Research agent: “What’s the best library for [X] in [stack]?” — Developer agent: “Generate boilerplate for [component] matching our architecture” — Review agent: “Review this PR for security and performance issues” 11:00 milestone check: backend serving data? Can the frontend fetch from it? - Integration point: coordinate API contracts early — the Module 4 spec-driven lesson pays off here	

12:00	Working Lunch	45 min
	Keep building. Eat at your desk if you want. Team stand-up during lunch: 2 min per person — what's done, what's next, what's blocking?	
12:45	Build Sprint Block 2: Integration	135 min
	- EXERCISE 5 (continued): Connect the pieces — Frontend → API integration: data flows end-to-end — Authentication (if needed): implement the simplest auth that works (even hardcoded for demo is OK at hackathon stage) — Deploy to staging: verify the integrated product works deployed, not just locally 14:00 milestone check: is one complete user flow working end-to-end? — If yes: start building the second feature — If no: cut scope. What is the minimum you need for tomorrow's user testing? - Scope management: the team lead (rotate this role) makes the call on what to cut vs. keep - Technical debt is OK — but track it. Create GitHub Issues labeled "hackathon-debt" for things you knowingly skip	
15:00	Break	15 min
	Stretch, clear your head	
15:15	Build Sprint Block 3: Feature Completion	75 min
	- EXERCISE 5 (continued): Polish the core, start the secondary features - Priority order: — Core user flow must be working, deployed, and demoable — At least basic error handling (no blank screens, no stack traces visible to users) — Data is realistic (seed with plausible data, not "test123") — One more feature if time allows 16:00 milestone: can you demo your product to someone who's never seen it?	
16:30	Day 2 Wrap-Up: Stand-Up & User Testing Prep	30 min
	- Team stand-up: what's working? What isn't? What's the plan for tomorrow? - User testing preparation for tomorrow afternoon: — EXERCISE 6: Write a user testing script (5–7 tasks for the tester to attempt) — Define success criteria for each task: what does the user need to accomplish? — Prepare observation sheet: what are you watching for? What questions will you ask after? — Review Steve Krug's "don't help the user" principle: watch them struggle, don't intervene - HOMEWORK: Continue building if needed (hackathon hours are flexible). Focus on making the core demo-ready for user testing tomorrow.	

Instructor Notes — Day 2: Day 2 is pure execution. Your job is to keep teams moving and prevent blocking. Do a physical walk-through of every team at each milestone (11:00, 14:00, 16:00). The most common Day 2 failure: teams spend too long on architecture and don't have working software by 14:00. If a team is behind at 14:00 and doesn't have end-to-end integration working, intervene directly: help them cut scope to the absolute minimum demonstrable product. The ClawSwarm integration varies: some teams will use it heavily, others will fall back to individual tool use. Both are valid — the learning is in trying, not in perfect adoption. Make sure every team has deployed to staging at least once by end of day. Code that only runs locally is not a prototype.

Day 3: 48-Hour Hackathon — Build Day 2 + User Testing

Theme: Complete the prototype, then put it in front of real users and learn from their reactions

Goal: Every team has a deployed prototype, has conducted at least 3 user testing sessions, has synthesized findings into actionable insights, and has a prioritized iteration plan.

Schedule

09:00	Morning Build Sprint	150 min
	• EXERCISE 5 (final push): Complete the prototype • The morning is the last opportunity for significant new code. Focus on: • — Completing the core user flow if not done yesterday • — Adding the one or two features that make the demo compelling • — Seeding realistic data (the difference between a demo and a toy) • — Basic styling (doesn't need to be beautiful, but shouldn't be obviously broken) • — Deploy to production (not just staging) by 11:00 • 10:00 hard milestone: if it's not working in production by now, enter emergency scope-cut mode • — Question: "What is the ONE thing we must show users to prove this concept works?" • — Cut everything else. Ruthlessly. • 11:00: code freeze for user testing. No new features. Only bug fixes from this point.	
11:30	User Testing Preparation	30 min
	• EXERCISE 7: Final user testing preparation • — Finalize user testing script (from last night's homework) • — Assign roles: facilitator (runs the session), note-taker (observes and documents), operator (handles tech issues) • — Set up screen recording if possible (with user consent) • — Prepare the environment: clear browser cache, reset demo data, test the deployment URL • — Dry run: one team member plays the user while others practice facilitating • Ground rules for user testing: • — Don't help the user. Let them struggle. Their confusion IS the data. • — Don't explain. If you have to explain it, the UI failed. • — Don't defend. If they criticize something, write it down. Don't argue. • — Ask "what are you thinking?" not "do you like it?"	
12:00	Lunch	30 min
	• Quick lunch — user testing starts at 12:30	

12:30	User Testing Sessions	120 min
	• EXERCISE 8: Conduct 3–5 user testing sessions (20–25 min each) • User pool: instructor-sourced external participants, students from other teams acting as users, or remote participants via video call • Each session follows the testing script: • — Brief intro: “We’re testing a prototype. We’re testing the product, not you. Think out loud.” • — Background questions: “How do you currently handle [the problem]?” (2 min) • — Task-based testing: ask the user to complete each task from the script (15 min) • — Wrap-up: “What was most confusing? What would make this useful for you? Would you use this?” (3 min) • Note-taker documents for each session: • — Task completion: which tasks succeeded? Which failed? Where did the user get stuck? • — Quotes: exact words the user said (especially moments of confusion or delight) • — Surprises: anything the team didn’t expect • — Severity: for each issue found, rate: Critical (blocks usage) / High (frustrating) / Medium (noticeable) / Low (cosmetic) • Rotate roles between sessions so everyone gets facilitation experience	
14:30	Break	15 min
	• Decompress after user testing — it can be humbling	
14:45	User Testing Synthesis Workshop	60 min
	• EXERCISE 9: Synthesize user testing findings as a team • — Affinity mapping: write every finding on a sticky note (physical or digital), cluster by theme • — Pattern identification: what issues appeared in 2+ sessions? Those are real patterns, not noise. • — Severity matrix: map findings by frequency (how many users hit it) vs. severity (how bad is it) • — Key insight summary: 3–5 sentences capturing the most important thing you learned from users • — Pivot or persevere decision: does the core concept work? Or do you need to rethink? • Write a User Testing Report (committed to repo): • — Methodology: how many users, what tasks, what format • — Key findings (top 5): each with evidence (quotes, task failure data) • — Recommendations: what to fix first based on severity and effort • This is the Build-Measure-Learn loop from The Lean Startup — applied in real-time	
15:45	Iteration Planning	45 min

	• EXERCISE 10: Create a prioritized iteration plan based on user feedback • — Sort findings into: Fix Now (critical, affects demo), Fix Tomorrow (high, improves experience), Fix Later (nice-to-have) • — Assign fixes to team members • — Update GitHub Projects board: add user testing issues, prioritize • — Decision: what are you changing for tomorrow's iteration? What are you NOT changing? (Write an ADR if it's a significant pivot) • Key principle: don't fix everything. Fix the things that make the demo compelling and the concept provable.
--	--

16:30	Day 3 Wrap-Up	30 min
	• Each team: 2-minute share — “The most surprising thing users told us was...” • Cross-pollination: did other teams' findings reveal anything about your own prototype? • HOMEWORK: Implement the “Fix Now” items. Deploy updates. Prepare for tomorrow's iteration and ClawSwarm deep dive.	

Instructor Notes — Day 3: Day 3's user testing is often the most transformative experience of the entire program for students who have never watched real users interact with something they built. The “don't help” and “don't defend” rules are hard to follow — facilitators instinctively want to rescue struggling users. Coach them to sit on their hands and take notes. Sourcing external test users is critical: students testing each other's products is a fallback, not the ideal. Try to arrange 2–3 external participants per team (from partner organizations, local meetup groups, or online panels). The synthesis workshop should feel energizing, not demoralizing — frame failures as discoveries. The teams that listen most honestly to user feedback produce the best iterations on Day 4.

Day 4: Iteration, Polish & Multi-Agent Deep Dive

Theme: Iterate on user feedback, polish the prototype for Demo Day, and go deeper into ClawSwarm's multi-agent architecture

Goal: Every team has an iterated prototype reflecting user feedback, every individual has a working ClawSwarm multi-agent configuration with 2+ specialized agents, and the demo is rehearsed and ready.

Schedule

09:00	Iteration Sprint	150 min
	- EXERCISE 11: Implement user feedback improvements — Work through the “Fix Now” and “Fix Tomorrow” items from yesterday’s iteration plan — Focus on the changes that make the demo most compelling: — The core user flow should feel smooth — no confusing navigation, no dead ends — Error states should show friendly messages, not crashes — Data should look realistic and relevant to the problem domain — Deploy incrementally: push every fix to production as it’s verified — Apply the Module 2 verification checklist even under time pressure — this is the habit being tested 10:30 milestone: has the user feedback been addressed? Is the prototype substantially better than yesterday? 11:00: feature freeze. Only bug fixes and polish from this point.	
11:30	ClawSwarm Multi-Agent Deep Dive (Individual Exercise)	75 min
	- EXERCISE 12: Build and document a comprehensive ClawSwarm multi-agent configuration - While the team prototype is in feature freeze, each student builds their own ClawSwarm setup: — Configure at least 2 specialized agents (e.g., Research + Developer, or Monitor + Reporter) — Implement cross-agent coordination: Agent A’s output feeds into Agent B’s task — Test with a real task: have the swarm complete a multi-step workflow end-to-end — Experiment with different LLM assignments: match models to workload — Lightweight model (Gemini Flash, Haiku) for simple tasks: summarization, formatting — Powerful model (Claude Opus, Gemini Pro) for complex reasoning: architecture, strategy — Document: cost vs. quality trade-offs for each model/task combination	

	— Implement shared memory: GOALS.md, DECISIONS.md, PROJECT_STATUS.md (from OpenClaw community patterns) — Push your ClawSwarm configuration and documentation to your individual portfolio repo - Comparison write-up (in design journal): — ClawSwarm (messaging-first, gRPC, Rust path) vs. Antigravity Manager View (IDE-first, artifacts) — When would you recommend each for a real team? — How does multi-agent orchestration change the Renaissance Developer's role?
--	---

12:45	Lunch	60 min
	Team lunch — discuss demo strategy	

13:45	Demo Preparation Workshop	60 min
	- EXERCISE 13: Prepare the Demo Day presentation - Demo format (each team gets 10 minutes): — Problem & user (1 min): who are you solving for? What's the pain? — Live demo (3 min): show the deployed prototype working end-to-end — User testing findings (2 min): what did you learn from real users? What changed? — Architecture & AI story (2 min): how did you build this? Which tools? What did verification catch? — ClawSwarm experience (1 min): how did multi-agent support the build? What worked? — What's next (1 min): if this were a real product, what would you build next? - Rehearsal: each team does a full run-through with timer — Designate a primary presenter and a backup — Pre-flight check: is the deployment stable? Is the demo data loaded? Does the network work? — Contingency: what do you show if the live demo breaks? (Screenshots, video recording)	

14:45	Break	15 min
	Deep breath before final polish	

15:00	Final Polish & Documentation Sprint	60 min
	- EXERCISE 14: Final deliverables completion - Team deliverables (in team repo): — README: problem statement, solution description, demo URL, setup instructions, team members — User Testing Report: methodology, findings, recommendations (from Day 3 Exercise 9)	

	• — Architecture sketch + ADRs (at least 1 significant trade-off documented) • — GitHub Projects board: all hackathon tasks tracked, issues from user testing labeled • Individual deliverables (in personal portfolio repo): • — ClawSwarm configuration with documentation and multi-agent comparison write-up • — Personal hackathon reflection: your role, what you contributed, what you learned about team delivery
--	---

16:00	Day 4 Wrap-Up	30 min
	• Final rehearsal if needed • Instructor check: every team has a stable deployment, a rehearsed demo, and all documentation committed • HOMEWORK: Get some rest. Review your demo one more time. Be ready for 09:00 sharp. • Preview: Demo Day tomorrow — the whole cohort judges, votes, and celebrates	

Instructor Notes — Day 4: Day 4 has a split personality: the morning is team iteration, the midday is individual ClawSwarm work, and the afternoon is demo prep. The feature freeze at 11:00 is critical — teams that keep adding features instead of polishing will have a worse demo. The ClawSwarm individual exercise (Exercise 12) ensures every student gets hands-on multi-agent experience even if their team used it lightly during the hackathon. The demo rehearsal is essential — unrehearsed hackathon demos consistently run over time, encounter technical failures, and miss the most important points. Push teams to rehearse at least twice with a timer. The biggest risk on Day 4 is teams exhausting themselves trying to add “one more thing” instead of making what they have excellent.

Day 5: Demo Day, Retrospective & Calibration

Theme: Present, judge, celebrate, and reflect on the shift from individual capability to team delivery

Goal: Every team has delivered a compelling demo, every student has participated in structured feedback, and the cohort has reflected on the growth from Modules 1–6.

Schedule

09:00	Demo Day: Hackathon Showcase	120 min
	• Each team: 10 minutes presentation + 5 minutes Q&A (strictly timed) • Judging criteria (all judges — instructors + peers — score 1–5 on each): • — Problem Clarity: was the problem well-defined and the user clearly identified? • — Solution Viability: does the prototype actually solve the stated problem? • — Technical Execution: is the code quality, architecture, and CI/CD solid for a 48-hour build? • — User Validation: did real user testing inform the solution? Are findings credible? • — Presentation Quality: was the demo clear, compelling, and within time? • — AI Tool Integration: was AI used effectively AND verified rigorously? • After all presentations: tallying and awards: • — Best Overall Prototype (highest combined score) • — Best Problem Discovery (sharpest problem definition and user empathy) • — Best User Testing (most rigorous validation and insightful synthesis) • — Best AI Integration (most effective use of AI tools with verification discipline) • — People's Choice (cohort vote)	
11:00	Break	15 min
	• Celebrate — you just completed a full design sprint in one week	
11:15	Hackathon Retrospective (Team + Whole-Cohort)	60 min
	• EXERCISE 15: Team retrospective (20 min in teams) • — What went well in our team collaboration? What was dysfunctional? • — How did we handle scope management? Did we cut early enough? • — How effectively did we use AI tools as a team (vs. individuals using tools separately)? • — What would we do differently if we had another 48 hours? • — Commit a retrospective document to the team repo • Whole-cohort retrospective (40 min): • — Each team shares their top learning (2 min per team) • — Cohort-wide patterns: what worked across teams? What failed across teams?	

	— The build-speed paradox: did AI tools make you faster AND better? Or faster at the cost of quality? — Discussion: what would need to change for this to work in a real company with real stakeholders?
--	---

12:15	Lunch	60 min
	Informal — cross-team discussions	

13:15	Personal Development Map Update & Module 7 Preview	60 min
	- EXERCISE 16: Personal Development Map v6 — Re-rate on five mindsets: how has Curiosity evolved (problem discovery)? How about Outcome orientation (user validation)? — Competency layers: Module 6 targeted Layer 4 (Strategist — Rapid Prototyping, Problem Discovery, Team Collaboration) — New dimension: how comfortable are you leading a team under pressure? What was your role this week? — Compare Day 1 vs. now: you started the program designing individual radar charts. This week you led (or contributed to) a team that delivered a validated prototype in 48 hours. - Instructor 1:1s through the afternoon - Module 7 preview: Business Immersion & Product Thinking — Next week shifts to the business side of the Renaissance Developer: Lean Canvas, customer discovery, P&L reading, business cases — This is where many engineers feel most uncomfortable — and where the program adds the most unique value	

14:15	Impossible Stuff Day: Self-Directed Exploration	105 min
	- Week 6 Impossible Stuff Day suggestions: — Extend your hackathon prototype into a production-ready application (apply Module 3–5 practices) — Build an advanced ClawSwarm configuration: 4+ agents with sophisticated routing logic — Compare ClawSwarm + Antigravity + individual tools: write a definitive comparison blog post — Create a “hackathon playbook”: document your team’s process as a reusable guide for future sprints — Read ahead: “The Lean Startup” Chapters 5–8 for Module 7’s business immersion — Read ahead: the full OpenClaw monorepo structure for Module 8’s enterprise architecture analysis — Write a blog post: “What 48 hours taught me about building with AI under pressure” - Share explorations in last 15 min	

16:00	Badge Ceremony & Week 6 Close	30 min
	• Badge ceremony: Rapid Builder badge for students who completed all deliverables • Team achievements: announce all awards from Demo Day judging • Individual achievements: best ClawSwarm config, best personal contribution to a team, best user testing facilitation • Read the Developer Manifesto together (recurring ritual) • 60% through the program. You can now build (M3), design (M4), ensure reliability (M5), and deliver under pressure with teams (M6). • The final stretch: business immersion (M7), organizational communication (M8), then the capstone (M9–10).	

Instructor Notes — Day 5: Demo Day is the week’s emotional high point. Strict timing is essential — 10 minutes per team, 5 minutes Q&A. Have a visible timer. The judging criteria should be explained before the first demo and visible throughout. Peer scoring keeps everyone engaged during other teams’ presentations. The awards ceremony should be celebratory and specific — call out what each winning team did well, not just that they won. The team retrospective often surfaces hard truths about collaboration: uneven contribution, communication failures, scope management arguments. These are valuable lessons — don’t sanitize them. The Module 7 preview is important: many technical engineers feel anxiety about the business immersion week. Frame it positively: “The business skills are what make you a Renaissance Developer instead of just a fast coder.”

Appendix A: Exercise Summary

#	Day	Exercise	Type
1	1	Problem Discovery: restate problem from user perspective, define boundary, write problem statement, define MVP	Discovery
2	1	Team Solution Design: silent sketch → present → merge → task breakdown → tech stack	Team Design
3	1	ClawSwarm setup: configure 2+ specialized agents, connect messaging, deploy in Docker	Setup
4	1	Team Architecture Sprint: quick design, 1 ADR, CI/CD pipeline, initial commits	Architecture
5	2–3	48-Hour Hackathon Build: full prototype from zero to deployed production	Build (Team)
6	2	Write user testing script: 5–7 tasks, success criteria, observation sheet	Preparation
7	3	User testing preparation: role assignment, environment setup, dry run	Preparation
8	3	Conduct 3–5 user testing sessions: facilitate, observe, document findings	User Research
9	3	Synthesize findings: affinity mapping, severity matrix, key insights, pivot/persevere	Analysis
10	3	Iteration planning: prioritize fixes, assign tasks, update projects board	Planning
11	4	Iteration sprint: implement user feedback fixes, deploy, verify	Build (Team)
12	4	ClawSwarm multi-agent deep dive: 2+ agents, cross-agent coordination, model matching, shared memory	Build (Individual)
13	4	Demo preparation: rehearse presentation, pre-flight checks, contingency plan	Presentation
14	4	Final polish: README, user testing report, ADRs, projects board, documentation	Documentation
15	5	Team + whole-cohort retrospective: collaboration, scope management, AI tool effectiveness	Reflection
16	5	Personal Development Map v6: team delivery dimension, problem discovery growth	Reflection

Appendix B: Deliverables Checklist

Complete all items to earn the Rapid Builder badge:

1. **Team Prototype** — Working application deployed to production, built in 48 hours, with CI/CD pipeline via GitHub Actions and GitHub Projects board tracking all work
2. **Problem Statement** — User-centered problem definition with boundary, MVP scope, and solution design, committed before build started
3. **User Testing Report** — Methodology, 3–5 session findings with quotes and task completion data, severity matrix, key insights, and recommendations
4. **Iteration Evidence** — Documented changes made based on user feedback, with before/after comparison and ADR for any significant pivots
5. **ClawSwarm Configuration** — Individual ClawSwarm setup with 2+ specialized agents, cross-agent coordination, model matching documentation, and comparison write-up (ClawSwarm vs. Antigravity)
6. **Team Retrospective** — Documented retrospective covering collaboration, scope management, AI effectiveness, and specific learnings, committed to team repo
7. **Personal Hackathon Reflection** — Individual write-up of your role, contributions, what you learned about team delivery under pressure, committed to portfolio repo
8. **Demo Presentation** — Delivered within 10-minute time limit, covering: problem, demo, user testing, architecture, AI story, and next steps
9. **Personal Development Map v6** — Updated with team delivery dimension, problem discovery growth, and evidence from Module 6

Appendix C: Hackathon Problem Bank — Instructor Guide

For instructors: Curate 6–8 problems for the hackathon. The best problems are real business pains that are ambiguous enough to require Problem Discovery but bounded enough for a 48-hour prototype.

Problem Selection Criteria

- **Ambiguous but bounded:** the problem is clear but the solution is not. Students must discover the right approach.
- **User-centric:** there is a specific, identifiable user whose life improves if the problem is solved.
- **Technically achievable:** a team of 3–4 mid-level engineers with AI tools can build a credible prototype in 48 hours.
- **AI-relevant:** the solution benefits from AI integration (RAG, summarization, automation, analysis) but is not purely an AI demo.
- **Testable with real users:** you can put the prototype in front of someone and observe whether it helps.

Sample Problems

1. **CRM Call Summary:** “Our sales team wastes 3 hours/day updating CRM after customer calls. They forget details and the data quality is terrible.” Domain: Sales/CRM. AI angle: conversation summarization, auto-form-fill. User: sales representatives.
2. **Onboarding Knowledge:** “New hires take 6 weeks to become productive because institutional knowledge is scattered across Confluence, Slack, and people’s heads.” Domain: HR/Knowledge Management. AI angle: RAG over internal docs, contextual Q&A. User: new hires.
3. **Manufacturing Quality:** “Quality issues in manufacturing are caught too late because inspection data lives in spreadsheets and nobody looks at trends.” Domain: Manufacturing/Operations. AI angle: anomaly detection, trend visualization, alerting. User: quality managers.
4. **Meeting Overload:** “Our engineering managers attend 25+ meetings/week and can’t keep track of decisions, action items, and who committed to what.” Domain: Engineering Management. AI angle: meeting summarization, action item extraction, follow-up tracking. User: engineering managers.
5. **Customer Support Routing:** “40% of support tickets are routed to the wrong team, causing 2-day average resolution delay.” Domain: Customer Support. AI angle: intent classification, smart routing, escalation prediction. User: support agents and managers.
6. **Compliance Documentation:** “Our compliance team spends 60% of their time manually gathering evidence for audits from 12 different systems.” Domain: Governance/Compliance. AI angle: automated evidence collection, gap analysis, report generation. User: compliance officers.

7. **Developer Productivity:** “Our developers spend 30% of their time searching for how things work because our documentation is always out of date.” Domain: Developer Experience. AI angle: auto-generated docs from code, intelligent code search, context-aware Q&A. User: software developers.
8. **Supply Chain Visibility:** “We don’t know a supplier is going to miss a delivery until the day it was due, and by then it’s too late to find alternatives.” Domain: Supply Chain. AI angle: predictive analytics, risk scoring, alternative supplier matching. User: procurement managers.

Sourcing Real Problems: Ideally, problems should come from partner organizations who will also provide test users on Day 3. This makes the hackathon feel like real forward-deployed engineering. Reach out to partner companies 4–6 weeks before Module 6 to source 2–3 real business challenges with committed user test participants.

End of Module 6 Lesson Plan